Practical 3-4 - Introduction to optimization

Xiru ZHANG

Year 2023/2024

Contents

1 Exercise 12 (Golden section search)

1.1 Implementation of Golden section search

Listing 1: Golden section search Implementation in MATLAB

```
function [xmin, fmin, iterations] =
   golden_section_search(func, a, b, accuracy)
    max_iterations = 100;
    golden_ratio = (1 + sqrt(5)) / 2;
    x1 = a;
    x4 = b;
    x2 = x4 - (x4 - x1) / golden_ratio;
    x3 = x1 + (x4 - x1) / golden_ratio;
    % Evaluate the function at initial points
    f1 = func(x1);
    f2 = func(x2);
    f3 = func(x3);
    f4 = func(x4);
    % Start iterations
    iterations = 0;
    while (x4 - x1) > accuracy && iterations <
       max_iterations
        iterations = iterations + 1;
        if f2 < f3
            x4 = x3;
            x3 = x2;
            x2 = x4 - (x4 - x1) / golden_ratio;
            f4 = f3;
            f3 = f2;
            f2 = func(x2);
        else
            x1 = x2;
            x2 = x3;
            x3 = x1 + (x4 - x1) / golden_ratio;
            f1 = f2;
            f2 = f3;
            f3 = func(x3);
        end
    end
    if f2 < f3
        xmin = x2;
        fmin = f2;
    else
        xmin = x3;
        fmin = f3;
```

```
    end
end
```

1.2 Implementation of Newton's Method

Listing 2: Newton's Method Implementation in MATLAB

```
function [root, iterations] = newtons_method(func,
   derivative, x_0, err, max_iterations)
    x0 = x_0;
    iterations = 0;

    while true
        iterations = iterations + 1;
        f = func(x0);
        df = derivative(x0);
        if df == 0
            error('Derivative␣is␣zero.␣No␣solution␣found
               .');
        end

        x1 = x0 - f / df;
        if abs(x1 - x0) < err || iterations >=
           max_iterations
            break;
        end
        x0 = x1;
    end
    root = x1;
    if iterations >= max_iterations
        warning('Maximum␣iterations␣reached.␣Solution␣
           may␣not␣have␣converged.');
    end
end
```

1.3 Test

1.3.1 $f(x) = \sin(x)$ on $[0, pi/2]$

- The **Golden Section Search** method converged to the root at $x = 3.2249 \times 10^{-7}$ after 30 iterations.
- The **Newton's Method** found the root at $x = 0$ in 5 iterations.
- The result obtained from **fminbnd** was $x = 6.4177 \times 10^{-5}$, the function's value at this point is very close to its minimum on the given interval, which is $\sin(0) = 0$.

1.3.2 $f(x) = (\arctan(x))^2$ on $[-1, 1]$

- The **Golden Section Search** method required **31 iterations** to find that the minimum occurs at $x = -1.2688 \times 10^{-7}$.
- **Newton's Method** found the root of the function's derivative to be approximately $x = 7.8615 \times 10^{-7}$ after **19 iterations**.
- MATLAB's `fminbnd` function, utilized for bounded optimization, computed the minimum location to be $x = \mathbf{2.7756 \times 10^{-17}}$.

1.3.3 $f(x) = |\ln(x)|$ on $[1/2, 4]$

- The **Golden Section Search** method required **32 iterations** to find an optimal solution. The minimum function value reported as 1 is inconsistent with the theoretical minimum value of 0, which occurs at $x = 1$.
- **Newton's Method** is not applicable for this function as it requires the first derivative of the function to find zeros. The derivative of $f(x)$ is $\frac{1}{x}$ for $x > 1$ and $-\frac{1}{x}$ for $x < 1$, neither of which equals zero within the interval $[1/2, 4]$. Additionally, the function is not differentiable at $x = 1$ where the minimum value occurs, due to the absolute value operation creating a cusp. This makes Newton's method unsuitable for finding the minimum of $f(x)$.
- MATLAB's `fminbnd` function yielded an optimal solution of 1.0000 with a function value of approximately 9.4937×10^{-6}, which is consistent with the expected theoretical minimum.

1.3.4 $f(x) = |x|$ on $[-1, 1]$

- The **Golden Section Search** method required **31 iterations** to locate the optimum. It reaches its minimum at $x = -1.2688 \times 10^{-7}$.
- Newton's method was not used because it is not suitable for functions that are not differentiable at their optimum points, as is the case with $f(x) = |x|$ at $x = 0$.
- MATLAB's `fminbnd` function found the minimum at $x = 2.7756 \times 10^{-17}$, effectively zero within machine precision. This is consistent with the known analytical minimum of $f(x)$ at $x = 0$.

2 Exercise 13

2.1 Gradient and Hessian

$$g(x) = \begin{bmatrix} 2x_1 - 400x_1(-x_1^2 + x_2) - 2 \\ -200x_1^2 + 200x_2 \end{bmatrix}$$

$$H(x) = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix}$$

2.2 Local Minimum Verification at $\mathbf{x}^* = [1, 1]^T$

The gradient at $\mathbf{x}^*$ is:

$$g(\mathbf{x}^*) = \mathbf{0},$$

and the Hessian at $\mathbf{x}^*$ is:

$$H(\mathbf{x}^*) = \begin{bmatrix} 802 & -400 \\ -400 & 200 \end{bmatrix}.$$

The two eigenvalues of the matrix are approximately 4 and 1001.6. The eigenvalues of $H(\mathbf{x}^*)$ are positive, which means that $H(\mathbf{x}^*)$ is positive definite, and therefore, $\mathbf{x}^*$ is a local minimum of f.

2.3 Newton's Method Iteration for Rosenbrock Function

Given the initial point $\mathbf{x}_0 = (-1, -2)^T$, the first five iterates of Newton's method for minimizing the Rosenbrock function are:

$$\begin{aligned}
\text{Initial point:} &\quad (-1.000000, -2.000000) \\
\text{Iteration 1:} &\quad \mathbf{x}_1 = (-0.996672, 0.993344) \\
\text{Iteration 2:} &\quad \mathbf{x}_2 = (0.995587, -2.977904) \\
\text{Iteration 3:} &\quad \mathbf{x}_3 = (0.995593, 0.991205) \\
\text{Iteration 4:} &\quad \mathbf{x}_4 = (1.000000, 0.999981) \\
\text{Iteration 5:} &\quad \mathbf{x}_5 = (1.000000, 1.000000)
\end{aligned}$$

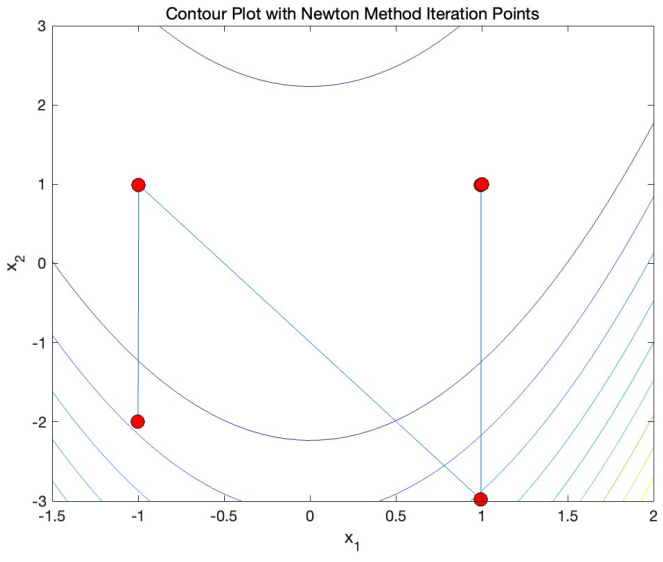

Figure 1: Newton's Method Iteration for Rosenbrock Function

2.4 Convergence Analysis of Newton's Method

The norms of the error at each iteration are calculated as $\|x - x^*\|$, where $x^* = (1, 1)$ is the optimal solution. The error norms for the given iterates are:

$$\begin{aligned}
\|x_0 - x^*\| &\approx 3.6055 \\
\|x_1 - x^*\| &\approx 1.9967 \\
\|x_2 - x^*\| &\approx 3.9779 \\
\|x_3 - x^*\| &\approx 0.0098 \\
\|x_4 - x^*\| &\approx 0.000019 \\
\|x_5 - x^*\| &= 0
\end{aligned}$$

To determine if the convergence is quadratic, we would expect the ratio $\|e_{k+1}\|/\|e_k\|^2$ to be approximately constant. The ratios for the provided iterates are:

$$\begin{aligned}
\text{From initial to iteration 1:} &\quad \approx 0.1536 \\
\text{From iteration 1 to 2:} &\quad \approx 0.9978 \\
\text{From iteration 2 to 3:} &\quad \approx 0.0006217 \\
\text{From iteration 3 to 4:} &\quad \approx 0.1963
\end{aligned}$$

3 Exercise 14

3.1 Implementation

α	Final x	Final Function Value
0.1	$(0, 0)$	0
0.5	$(0, -1)$	2
1.0	NaN	NaN

Table 1: Summary of Results for Different Step Sizes

For $\alpha = 0.1$, the algorithm converges smoothly to the global minimum at $(0, 0)$.

With $\alpha = 0.5$, the algorithm exhibits oscillatory behavior in the x_2 component, failing to converge. This suggests that the step size is too large, causing the algorithm to "overshoot" the minimum and enter a cycle.

For $\alpha = 1.0$, the result is NaN, indicating numerical instability in the iteration process, likely due to an excessively large step size.

Listing 3: Gradient descent Implementation in MATLAB

```
function [x, fval, history] = gradient_descent(alpha,
   max_iter, tol, func, grad_func, x0)
    x = x0;
```

```
    fval = func(x);
    history = [];
    for iter = 1:max_iter
        grad = grad_func(x);
        x = x - alpha * grad;
        fval = func(x);
        history = [history; x', fval];
        if norm(grad) < tol
            break;
        end
    end
end
```

3.2 Rosenbrock function

Step Size α	Final Point x	Function Value
0.001	$(-0.135065, 0.021803)$	1.289639

Table 2: Results of the Gradient Descent after 1000 Iterations

The small step size $\alpha = 0.001$ used in this experiment may have been too conservative, resulting in slow convergence.

3.3 Optimization of Rosenbrock Function with Wolfe's Method

The algorithm was tested for various combinations of m_1 and m_2. The results demonstrate the impact of these parameters on the convergence of the optimization process:

m_1	m_2	Final Point	Function Value
0.010	0.800	$(1.389932, 1.928695)$	0.153082
0.010	0.900	$(1.389932, 1.928695)$	0.153082
0.010	0.990	$(1.389932, 1.928695)$	0.153082
0.100	0.800	$(1.006462, 1.012953)$	0.000042
0.100	0.900	$(1.006462, 1.012953)$	0.000042
0.100	0.990	$(1.006462, 1.012953)$	0.000042
0.200	0.800	$(1.008264, 1.016596)$	0.000068
0.200	0.900	$(1.008264, 1.016596)$	0.000068
0.200	0.990	$(1.008264, 1.016596)$	0.000068

Table 3: Optimization Results with Different Wolfe Parameters

Each combination was run for a maximum of 1000 iterations, indicating a potential limit set by the maximum number of iterations or an early stopping condition based on the algorithm's tolerance level.

4 Exercise 15

4.1 Nelder-Meade algorithm

Listing 4: Nelder-Meade algorithm Implementation in MATLAB

```
function [xmin, fval, iter] = nelder_mead(f, x0,
   max_iter, tol)
    alpha = 1;
    gamma = 2;
    rho = 0.5;
    sigma = 0.5;
    n = length(x0);
    simplex = repmat(x0(:).', n+1, 1);
    for i = 2:n+1
        simplex(i,i-1) = simplex(i,i-1) + 1;
    end
    fvals = arrayfun(@(i) f(simplex(i,:)), 1:n+1);
    [fvals, idx] = sort(fvals);
    simplex = simplex(idx, :);
    iter = 0;
    while iter < max_iter
        centroid = mean(simplex(1:end-1,:), 1);
        xr = centroid + alpha * (centroid - simplex(end
           ,:));
        fr = f(xr);
        if fr < fvals(1)
            xe = centroid + gamma * (xr - centroid);
            fe = f(xe);
            if fe < fr
                simplex(end,:) = xe;
                fvals(end) = fe;
            else
                simplex(end,:) = xr;
                fvals(end) = fr;
            end
        elseif fr < fvals(end-1)
            simplex(end,:) = xr;
            fvals(end) = fr;
        else
            if fr < fvals(end)
```

```
                xc = centroid + rho * (simplex(end,:) -
                   centroid);
                fc = f(xc);
                if fc < fvals(end)
                    simplex(end,:) = xc;
                    fvals(end) = fc;
                end
            else
                for i = 2:n+1
                    simplex(i,:) = simplex(1,:) + sigma
                       * (simplex(i,:) - simplex(1,:));
                    fvals(i) = f(simplex(i,:));
                end
            end
        end
        [fvals, idx] = sort(fvals);
        simplex = simplex(idx, :);
        if abs(fvals(1) - fvals(end)) < tol
            break;
        end
        iter = iter + 1;
    end
    xmin = simplex(1,:);
    fval = fvals(1);
end
```

4.2 Test the Rosenbrock function

The results obtained were as follows:

- Minimum found at: $[1.00009427342648, 1.00021156890486]$
- Function value at minimum: 6.1848×10^{-8}
- Number of iterations: 109

4.3 Compare with fminsearch

The results using MATLAB's `fminsearch` function were:

- `fminsearch` found the minimum at: $[0.999999869473974, 0.999999754728729]$
- `fminsearch` function value at minimum: 4.194×10^{-14}

The MATLAB's `fminsearch` function yielded a more accurate minimum, closer to the theoretical minimum of $(1, 1)$, and with a lower function value, indicating a more precise convergence compared to the custom implementation of the Nelder-Mead algorithm.